Slovak University of Technology

Faculty of Informatics and Information Technologies

Ilkovičova 3, 842 16 Bratislava 4

Message Transmission Using the UDP Protocol

Author: Michal Kebis

Study Program: Computer Systems and Networks

Year: 2, Exercise: Monday 11:00

Course: Computer Networks 1

Academic Year: 2008/09

Assignment:

On top of the UDP (User Datagram Protocol) of the transport layer of the TCP/IP network model, design and implement a protocol for message transmission between applications on computers (sending and receiving messages – "chat" functionality). The message length is not limited. The receiving application must be able to receive a message at any time, i.e., the receiving side does not need to be switched to receive mode in advance.

If necessary, the message will be transmitted in blocks, where the maximum block size should be derived from the maximum length of the data field of an Ethernet frame.

The UDP protocol is unreliable, therefore the designed protocol format should also include a checksum field and fields for possible communication reliability using a block ARQ method. Use the checksum used by the UDP protocol (RFC 768). Detection of an incorrect checksum on the receiving side will only be indicated. The user interface must allow:

- writing a message and selecting the destination computer on which the receiving application is running,
- displaying the received message and the IP address of the computer from which the message was sent,
- displaying checksums:
    - sent on the sender side,
    - received and calculated on the receiver side.

The designed and implemented solution must meet the following requirements:

1. Message transmission between applications on different computers using a checksum.
2. Division of the message into suitable blocks.
3. Error indication in case of detection on the receiver side. Communication errors will be simulated.

The documentation must include:

- specification of the designed protocol format with a description of individual fields and protocol operation,
- choice of transmitted block size,
- choice of possible error protection and acknowledgment method,
- description of the method for simulating error occurrence,
- block diagram of the solution,
- description of the user interface,
- choice of implementation environment.

Format of the UDP packet data part:

Type (1B)	Length (2B)	Flags (1B)	ID (2B)	Number (2B)	Total (2B)	Checksum (2B)
Data content depending on the Type (max 1460B)						

Type (1B) – Type of transmitted data:

0	Nickname
1	Message
2	ARQ data

Length (2B) – Number of transmitted characters in the main data part

Flags (1B):

xxxxxxx1	Fragment	indicates the message was split into multiple parts that must be reassembled
xxxxxx1x	Acknowledge	receipt confirmation, normally not used, intended for ARQ
xxxxx1xx	Ack #	inverted with each sent packet to distinguish ACKs

ID (2B) – Message identification number, incremented after each completely sent message

Number (2B) – Fragment number if the message does not fit into one frame

Total (2B) – Total number of message segments

Checksum (2B) – Checksum calculated from the custom header and transmitted data using the same method as the UDP checksum

Data (max 1460B) – Transmitted data (nickname, message, message fragment, or ARQ data)

Problem Description, Analysis and Design:

From the assignment it follows that we will use UDP (User Datagram Protocol) encapsulated in IP (Internet Protocol) within the TCP/IP network model.

TCP/IP Layer	Model
4. Application Layer	Message transmission
3. Transport Layer	UDP
2. Network Layer	IP
1. Network Interface Layer	Ethernet II

The UDP protocol is unreliable, connectionless, without acknowledgments and without guaranteed packet order.

Reliability of transmission will be ensured using a simple checksum of the data part. This will be implemented by padding with a zero byte to obtain an even number of octets, then summing pairs of octets. If the sum exceeds 16 bits, it is split into two 16-bit parts which are added together, and a one's complement is applied. The result is written into the header. In case of error detection on the receiving side (mismatch between transmitted and calculated checksum), this will be indicated by displaying a message in the receiver's main window. The sender will not be informed about the error, as retransmission of erroneous packets is not required by the assignment. Error simulation will be provided by generating an incorrect checksum when sending a message (via a checkbox).

Acknowledgment will not be implemented, but the solution will allow for future extension and will be described in the documentation. It will use the Stop-and-Wait ARQ method. The sender sends one message and waits for acknowledgment. If none is received within a certain time, the message is retransmitted. ACK packets use type 2 and the ACK flag. The ACK# flag is inverted to avoid duplicate acknowledgments.

The maximum message length that fits into one packet is derived from the maximum Ethernet frame payload. The data field is 1500 B, minus 20 B for IP, 8 B for UDP, and 12 B for the custom protocol header, resulting in 1460 B.

Communication will be peer-to-peer, without a server. All participants are equal and act as both client and server. Each computer stores its own participant data.

The user interface will be graphical, implemented using MFC in Visual C++. It will allow sending and receiving messages, managing IP address contacts, selecting recipients, simulating and detecting transmission errors. The contact list will be implemented using a simple linked list.